OPLURIX · ECOLOGICAL INTELLIGENCE

AN OPLURIX PRODUCT · VERSION 1.0 · EN

Biodiversity Data *Harmonizer*

Merge, clean, standardise, and validate biodiversity datasets from multiple sites, seasons, methods, and observers into a single, analysis-ready, MRV-compatible data structure.

BUILT FOR

- Multi-Site Research Projects
- Conservation NGO Data Officers
- Long-Term Monitoring Programmes
- REDD+ and MRV Data Managers

THE TRANSFORMATION

From "I have data from six sites, four observers, three seasons, and two methods — in five different formats — that I cannot combine for any analysis" to "I have one unified, validated, documented dataset that any analyst, funder, or MRV auditor can work with."

Haja Fabrice Razafindrabe Maminiaina
Field Biologist · Conservation Technologist · Founder, OPLURIX

v1.0 · 2026
OPLURIX · $99

HOW TO USE THIS PRODUCT

Read Sections 1–3 once to understand the problem and the system. Then work through Section 4 (the 8-Step Pipeline) with your actual datasets. The Schema Library (Section 6), Cleaning Rules (Section 7), and AI Prompt Pack (Section 8) are your working references throughout. The Validation Framework (Section 9) runs at the end of every harmonization cycle.

SECTION 1

The Core Promise

Multi-site, multi-season, multi-method biodiversity data is the most powerful kind of conservation data — and the hardest to use. A single-site, single-season camera trap dataset is clean by default: one observer, one format, one set of conditions. Add a second site with a different team, a second season with updated protocols, and a third method with its own file structure, and you have created a data integration problem that routinely takes researchers weeks or months to resolve.

The integration problem is not analytical. You do not need a more sophisticated model. You need the data in a consistent structure first. Species names spelled differently across datasets, dates in different formats, GPS coordinates in different datums, count data mixed with presence-absence data, observer codes that are not in any shared codebook — these are data quality problems, not statistical problems. They must be solved before any analysis begins.

This guide gives you a repeatable, documented pipeline for solving them. It is built on the actual integration challenges that arise when combining mammal transect data, herpetofauna visual encounter surveys, camera trap records, and acoustic monitoring outputs into a single harmonised dataset — the exact data types used in the ERIS biodiversity scoring system.

THE TRANSFORMATION

Before: "I have data from six sites, four observers, three seasons, and two methods — in five different formats — that I cannot combine for any analysis."

After: "I have one unified, validated, documented dataset with full provenance that any analyst, funder, or MRV auditor can work with."

SECTION 2

10 Real Pain Points

01 The same species is spelled differently across datasets.

"Lemur catta", "L. catta", "Lc", "Ring-tailed lemur", "LEMURCATTA" — five records of the same species that a join query treats as five different species. Resolving this across 3,000 records without a systematic approach takes days and introduces errors.

02 Date and time formats are inconsistent across sites and seasons.

Site 1 uses DD/MM/YYYY. Site 2 uses MM-DD-YY. Season 3 has times in 12-hour format. Season 1 has no times at all. A merged dataset with mixed date formats will produce silent errors in any temporal analysis — the wrong records will be sorted, filtered, and grouped.

03 GPS coordinates are in different formats or datums.

Degrees-minutes-seconds from one GPS unit. Decimal degrees from a mobile app. A third observer using UTM. Some coordinates entered without hemisphere (assuming local context). Spatial analysis on a merged dataset with mixed coordinate systems produces nonsense results with no error message.

04 Count data and presence-absence data are mixed.

A camera trap record says "1 individual detected." A transect record says "species present." An acoustic record says "7 calls recorded." These are different types of data that require different analytical treatments — but in a merged spreadsheet, they sit in the same "count" column and look identical.

05 Observer codes are not in any shared codebook.

"HFR", "Haja", "H. Fabrice", "Observer 1", and "HajaF" may all be the same person — or they may not. Without a shared observer registry, observer effects cannot be modelled and data cannot be attributed to a specific person for quality control purposes.

06 Detection distance data is missing or inconsistent.

Line transect distance sampling requires perpendicular distances to each detection. One observer estimated distances in metres. Another used categories (0-5m, 5-25m, 25-50m). A third did not record distances at all. These cannot be merged into a single distance-sampling analysis without imputation or exclusion — both of which require explicit decisions that must be documented.

07 Effort data is recorded at different spatial and temporal scales.

Camera trap effort is in trap-nights per station. Transect effort is in kilometres walked. Acoustic effort is in recording-hours per unit. These are not convertible to a single effort metric — but they must all be attached to their respective detection records for detection probability modelling to work.

08 Sensitive location data is mixed with shareable data in the same file.

Threatened species nest sites and exact den locations in the same dataset as standard vegetation plot coordinates. Before sharing the dataset with any external party, every sensitive location must be identified, flagged, and either removed or offset. This is not a data cleaning step — it is an ethical obligation.

09 **The merge produces duplicate records that are hard to detect.**

When the same observation is entered twice — once in a field app and once in a paper datasheet that was then digitised — the duplicate does not look like a duplicate. It has slightly different times, slightly different coordinates, and slightly different observer entries. Automated duplicate detection misses it. Visual inspection across thousands of records misses it too.

10 **The merged dataset is undocumented — nobody knows how it was created.**

Six months after the merge, the researcher who did it has moved on. The dataset exists but its provenance — which source files were merged, what cleaning decisions were made, what was excluded and why — exists only in one person's memory. An auditor, a new team member, or a journal reviewer cannot verify anything. This guide ends that problem by building documentation into every step.

SECTION 3

10 Desired Outcomes

#	Outcome
01	Merge datasets from multiple sites, seasons, methods, and observers into a single, analysis-ready file.
02	Resolve species name inconsistencies systematically using a controlled vocabulary and a species registry.
03	Standardise all dates, times, coordinates, and numeric fields to a single, consistent format.
04	Distinguish between count data, presence-absence data, and index data in the same unified dataset.
05	Flag and handle sensitive location data before any external sharing.
06	Detect and resolve duplicate records, both exact and near-duplicate.
07	Attach survey effort to every detection record in a form suitable for detection probability modelling.
08	Produce a harmonization log that documents every cleaning decision made.
09	Align the harmonized dataset with Darwin Core standards for GBIF submission or MRV reporting.
10	Pass a six-point validation framework that confirms the dataset is ready for analysis and external sharing.

SECTION 4

The 8-Step Harmonization Pipeline

Every step produces a documented output and feeds into the next. Do not skip steps. The sequence is designed so that errors are caught at the lowest possible cost — before they are baked into an analysis that takes weeks to run.

01 **Dataset Inventory** — Catalogue every source file before touching any data.

↓

02 **Master Schema Design** — Define the unified column structure before any merge.

↓

03 **Registry Building** — Build shared species, observer, and site registries.

↓

04 **Individual Dataset Cleaning** — Clean each source dataset against the master schema before merging.

↓

05 **Merge** — Combine the cleaned datasets. Preserve provenance columns throughout.

↓

06 **Cross-Dataset Cleaning** — Resolve inconsistencies that only appear after the merge.

↓

07 **Sensitive Data Handling** — Flag, offset, or remove sensitive location data.

↓

08 **Validation and Documentation** — Run the six-point validation framework. Write the harmonization log.

1

DATASET INVENTORY

Before touching any data, document every source file. This step takes 30–60 minutes and prevents the most common integration mistake: starting to clean data before understanding what you have.

INVENTORY TEMPLATE — ONE ROW PER SOURCE FILE

- → **file_id:** Short unique ID (e.g., "CT_IHOFA_2024")
- → **filename:** Exact filename including extension
- → **survey_method:** camera_trap / transect / acoustic / vegetation / other
- → **site:** Site name as it appears in the file
- → **date_range:** First and last observation date
- → **observer(s):** Names or codes as they appear in the file
- → **n_records:** Row count
- → **known_issues:** Any formatting problems, missing columns, or data quality issues already known
- → **column_names:** List of column headers exactly as they appear
- → **date_format:** How dates are recorded in this file
- → **coord_format:** GPS format used (decimal degrees / DMS / UTM / none)

Output: A dataset inventory CSV. This becomes part of the harmonization log.

2

MASTER SCHEMA DESIGN

Define the unified column structure that all source datasets will be mapped to. Every column has a name, a data type, a format, and a description. Columns not present in every source dataset are included with a defined NA handling rule.

Use the Schema Library in Section 6 to select and adapt the schema for your data types. Use Prompt 1 in the AI Prompt Pack to help draft the schema from your dataset inventory.

Output: A master schema document listing every column name, type, format, and NA rule. This is the reference against which all cleaning work is done.

3

REGISTRY BUILDING

Build three shared registries before cleaning any dataset. These registries are the controlled vocabularies that ensure consistency across all source files.

THREE REQUIRED REGISTRIES

- → **Species Registry:** Every species name that appears in any source dataset, mapped to: canonical Latin name, species code, taxonomic group, IUCN status, endemism status, sensitivity flag (sensitive location data required?)
- → **Site Registry:** Every site name that appears in any source dataset, mapped to: canonical site name, site code, region, GPS coordinates of site centroid, habitat type, protection status
- → **Observer Registry:** Every observer name or code that appears in any source dataset, mapped to: canonical observer code, full name, affiliation, experience level, seasons active

Output: Three registry CSV files. These are used throughout the cleaning steps and become part of the dataset documentation.

4

INDIVIDUAL DATASET CLEANING

Clean each source dataset separately before merging. The cleaning applies the Cleaning Rules from Section 7 and maps each source column to its master schema equivalent. Work through each dataset in isolation — do not try to clean and merge simultaneously.

For each source dataset, produce:

- → A cleaned version with all columns renamed to match the master schema
- → A source-specific cleaning log listing every change made and why
- → A column mapping table: source column name → master schema column name

Output: One cleaned CSV per source dataset + one cleaning log per source dataset.

5

MERGE

Combine all cleaned source datasets into a single file using a vertical merge (append rows). Do not join on columns at this stage — joins come after the merge is validated. Preserve provenance columns throughout: `source_file_id`, `source_row`, and `merge_date` on every record.

> THE PROVENANCE RULE — NON-NEGOTIABLE
>
> Every row in the merged dataset must be traceable back to a specific row in a specific source file. Never delete the provenance columns. An auditor must be able to look at any record in the harmonized dataset and verify it against the original source. Without provenance, the dataset cannot be audited.

Output: A single merged CSV with all records from all source datasets. Unchecked — cleaning continues in Step 6.

6

CROSS-DATASET CLEANING

Some inconsistencies only appear after the merge — species name variants that exist in different source files, site names that overlap or conflict across seasons, observer codes that collide. Run the cross-dataset cleaning rules from Section 7 Part B.

The three most common cross-dataset problems and how to handle them:

- → **Near-duplicate records:** Same species, same site, similar time, slightly different coordinates. Use Prompt 6 to detect these. Do not automatically delete — flag for expert review.
- → **Taxonomic name conflicts:** Species coded differently in two sites because different identification guides were used. Resolve using the Species Registry. Document every resolution in the harmonization log.
- → **Site boundary overlap:** Records at a coordinate that falls within two different site polygons. Assign to the site with the smaller area (most specific) and flag as SITE_AMBIGUOUS.

7

SENSITIVE DATA HANDLING

Before the dataset is shared with any external party — collaborator, funder, journal, data repository — sensitive location data must be handled. This is an ethical obligation, not a data cleaning step.

SENSITIVE DATA CATEGORIES FOR CONSERVATION DATASETS

- → **IUCN Critically Endangered or Endangered species:** Precise coordinates must be offset by ≥1 km in any public dataset
- → **Known nest, roost, or den sites:** Remove coordinates entirely from shared versions
- → **Poaching-sensitive species:** Any species with known commercial demand — orchids, reptiles, primates — apply coordinate generalisation regardless of IUCN status
- → **Community-sensitive locations:** Coordinates that would identify specific community members, farms, or settlements require consent before sharing

Maintain two versions of the dataset: the full internal version (all coordinates) and the shareable public version (sensitive coordinates offset or removed). Document which records were modified and why in the harmonization log.

8

VALIDATION AND DOCUMENTATION

Run the six-point Validation Framework from Section 9. Write the Harmonization Log from the template in Section 10. The dataset is not ready for analysis or sharing until both are complete.

Output: Validated harmonized dataset + harmonization log + metadata file. This package is the deliverable.

SECTION 5

Master Schema Design Principles

The master schema is the single most important document in the harmonization process. If it is wrong, every subsequent step produces a wrong result. Take the time to design it carefully before writing a single line of cleaning code.

Schema Design Rules

- **One column, one meaning.** A column called `notes` that contains observer comments, data quality flags, and behavioural observations is three columns poorly stored as one. Split it.
- **Controlled vocabulary for all categorical fields.** Every field that has a finite set of valid values must list those values in the schema. No free-text in categorical columns.
- **NA is not the same as 0 or empty.** Missing data, zero count, and not applicable are three different states. Your schema must handle all three explicitly.
- **Separate raw from interpreted.** Where a field contains a value that required human interpretation (species ID from a blurred camera trap image), maintain both the raw value and the interpreted value in separate columns.
- **Effort must be attached to detections.** Every detection record must be linkable to its corresponding effort record — the transect it was observed on, the camera trap station it was detected at, the acoustic unit that recorded it.
- **Timestamps must be timezone-aware.** If data comes from multiple countries or significant longitude ranges, include timezone offset in the timestamp format.

Column Type Definitions

Type code	Definition	Example values	Format
`TEXT`	Free text — use sparingly, only for notes and provenance fields	"Equipment slightly displaced on retrieval"	No length limit, UTF-8
`CODE`	Controlled vocabulary — must match registry	"LEMURCATTA", "HFR", "IHOFA_NE"	ALLCAPS, underscores, no spaces, no special chars
`DATE`	Calendar date	"2024-03-06"	ISO 8601: YYYY-MM-DD
`TIME`	Time of day	"07:42"	HH:MM, 24-hour, local time
`DATETIME`	Combined date and time	"2024-03-06T07:42:00"	ISO 8601 with T separator
`LAT`	Latitude in decimal degrees, WGS84	-18.45760	Negative for South, 5 decimal places
`LON`	Longitude in decimal degrees, WGS84	47.18723	Positive for East, 5 decimal places
`INT`	Integer — for counts, trap-nights, page numbers	3, 47, 0	No decimals, no negatives for counts
`FLOAT`	Decimal number — for measurements, distances, percentages	4.5, 87.3, 0.92	Up to 4 decimal places
`BOOL`	Binary flag	TRUE / FALSE	Only these two values; not 1/0 or Y/N
`FLAG`	Status or quality flag from controlled vocabulary	"HIGH", "ILLEGIBLE", "SPATIAL_OUTLIER"	From defined flag vocabulary

SECTION 6

Schema Library — 6 Biodiversity Data Types

Complete master schemas for the six most common conservation field data types. Use these directly or adapt them for your specific project. Every schema includes provenance columns and is Darwin Core compatible.

Schema 1 — Wildlife Detection (Camera Trap, Transect, Opportunistic)

MASTER SCHEMA — WILDLIFE DETECTION RECORDS

Column name	Type	Description / valid values
record_id	TEXT	Globally unique. Format: [source_id]_[row]. e.g. CT_IHOFA_2024_0042
source_file_id	CODE	From dataset inventory. Links record to source file.
survey_method	CODE	CAMERA_TRAP / LINE_TRANSECT / POINT_COUNT / OPPORTUNISTIC / FOCAL_FOLLOW
detection_date	DATE	YYYY-MM-DD. ILLEGIBLE if unreadable.
detection_time	TIME	HH:MM 24h. MISSING if not recorded.
site_code	CODE	From Site Registry. Canonical site code.
station_id	CODE	Camera station, transect, or plot ID within the site.
latitude	LAT	Decimal degrees WGS84. ILLEGIBLE / MISSING as applicable.
longitude	LON	Decimal degrees WGS84.
coord_original	TEXT	GPS entry as written in original record.
species_code	CODE	From Species Registry. UNID_[TAXON] if unidentified.
species_latin	TEXT	Full binomial. "cf." prefix for uncertain ID.
species_raw	TEXT	Exactly as written in source record.
count	INT	Number of individuals. ILLEGIBLE if unreadable.
count_min	INT	For range counts. NA if exact count recorded.
count_max	INT	For range counts. NA if exact count recorded.
detection_type	CODE	COUNT / PRESENCE_ABSENCE / INDEX / TRACK / CALL / SCAT
age_class	CODE	AD / JUV / INF / UNKNOWN
sex	CODE	M / F / UNKNOWN
distance_m	FLOAT	Perpendicular distance in metres (transects only). NA for other methods.
observer_code	CODE	From Observer Registry. Canonical observer code.
effort_link_id	TEXT	Links to effort record in the effort table.

sensitivity_flag	FLAG	PUBLIC / RESTRICTED / OFFSET_APPLIED. Set per species from Species Registry.
confidence	FLAG	HIGH / MEDIUM / LOW / ILLEGIBLE
notes	TEXT	Free text. Transcriber notes, observer comments, QC flags.
merge_date	DATE	Date this record entered the harmonized dataset.

Schema 2 — Survey Effort Table

EFFORT RECORDS — ONE ROW PER SURVEY UNIT PER DAY

Column name	Type	Description
effort_id	TEXT	Unique ID. Links to detection records via effort_link_id.
survey_method	CODE	Same controlled vocabulary as detection table.
site_code	CODE	From Site Registry.
station_id	CODE	Camera station, transect ID, or acoustic unit ID.
effort_date	DATE	Date of survey effort.
effort_start	TIME	Survey start time. MISSING if not recorded.
effort_end	TIME	Survey end time. MISSING if not recorded.
effort_unit	CODE	TRAP_NIGHT / KM_WALKED / RECORDING_HOUR / OBSERVER_HOUR
effort_value	FLOAT	Quantity of effort in the specified unit.
observer_code	CODE	From Observer Registry.
conditions	CODE	GOOD / FAIR / POOR. Standardised weather/visibility assessment.

Schemas 3–6 — Quick Reference

Schema	Data type	Key additional columns vs. Schema 1
Schema 3	Acoustic monitoring records	`unit_id`, `recording_file`, `freq_range_hz`, `call_type`, `n_calls`, `species_id_method` (MANUAL/BIRDNET/KALEIDOSCOPE), `id_confidence`
Schema 4	Vegetation plots	`plot_id`, `plot_size_m2`, `species_latin` (plant), `dbh_cm`, `height_m`, `cover_pct`, `stratum` (canopy/understorey/ground), `alive` (BOOL)
Schema 5	Herpetofauna VES records	`microhabitat`, `substrate`, `snout_vent_mm`, `tail_mm`, `mass_g`, `tissue_sample_id`, `photo_id`, `released_alive` (BOOL)
Schema 6	Drone survey detections	`flight_id`, `altitude_m`, `sensor_type` (THERMAL/RGB/MULTISPECTRAL), `image_file`, `detection_method` (MANUAL/YOLO_V8/OTHER), `bbox_confidence`, `ground_verified` (BOOL)

SECTION 7

40 Cleaning Rules

Organised by field category. Every rule has a trigger condition, the correct action, and a documentation requirement. Log every application of every rule in the cleaning log.

Part A — Within-Dataset Rules (apply before merge)

DATE AND TIME FIELDS

D1 — Standardise date to ISO 8601

`IF date field != YYYY-MM-DD → convert to YYYY-MM-DD`

Action: Convert. Log: original format, converted value, conversion method.

D2 — Separate combined date-time fields

`IF field contains both date and time → split into date and time columns`

Action: Split using the delimiter (space, T, comma). Log: original format.

D3 — Standardise time to 24-hour format

`IF time field uses 12-hour with AM/PM → convert to HH:MM 24h`

Action: Convert. Log ambiguous cases (no AM/PM indicator).

D4 — Flag date outside survey period

`IF detection_date outside [survey_start, survey_end] → flag TEMPORAL_OUTLIER`

Action: Flag. Do not delete. Log: flagged date and reason.

GPS AND COORDINATE FIELDS

G1 — Convert DMS to decimal degrees

`IF coordinate in D°M'S" format → convert to DD.DDDDD`

Action: Convert using formula. Preserve original in coord_original. Log conversion.

G2 — Convert DDM to decimal degrees

`IF coordinate in D°M.mmm' format → convert: DD = D + M.mmm/60`

Action: Convert. Preserve original. Log.

G3 — Infer hemisphere from study area

`IF latitude positive but study area is Southern Hemisphere → check for sign error`

Action: Flag as HEMISPHERE_CHECK. Do not automatically negate. Requires human confirmation.

G4 — Flag coordinate outside study area bounding box

`IF lat/lon outside [lat_min, lat_max, lon_min, lon_max] → flag SPATIAL_OUTLIER`

Action: Flag. Do not delete. Log: coordinates, bounding box, reason.

G5 — Flag coordinate precision below minimum

`IF decimal degrees has fewer than 4 decimal places → flag LOW_PRECISION`

Action: Flag. Retain value. Note that position accuracy is ±~10m minimum at 4 d.p.

G6 — Detect transposed coordinates

`IF latitude value is in longitude range AND longitude value is in latitude range → flag TRANSPOSED`

Action: Flag TRANSPOSED. Do not auto-swap — requires human confirmation. Log both values.

SPECIES NAME FIELDS

S1 — Map raw species name to Species Registry

`IF species_raw not in Species Registry → look up canonical name. If found: map. If not found: flag SPECIES_UNKNOWN.`

Action: Populate species_code and species_latin from registry. Preserve species_raw.

S2 — Handle "cf." identifications

`IF species name contains "cf.", "aff.", "?", or "probably" → prefix species_latin with "cf."`

Action: Set species_latin = "cf. [name]". Set confidence = MEDIUM or LOW depending on context.

S3 — Standardise unidentified records

`IF species not identified to species level → use UNID_[TAXON] format`

Example: UNID_LEMUR, UNID_FROG, UNID_SNAKE. Never leave species_code blank for unidentified records.

S4 — Flag taxonomic name changes

`IF species recorded under a synonym that has since changed → map to current accepted name`

Action: Use current accepted Latin name. Note old name in notes column. Log: original name, current name, source of change (e.g., ITIS, Catalogue of Life update).

COUNT AND NUMERIC FIELDS

N1 — Separate count ranges into min and max

`IF count field contains a range (e.g., "8-12", "~10", "10+") → split to count_min and count_max`

Action: Parse range. "10+" = count_min:10, count_max:NA. "~10" = count_min:8, count_max:12 with note "estimated".

N2 — Flag negative count values

`IF count < 0 → flag COUNT_NEGATIVE`

Action: Flag. Do not change value. Log: value and source row. Likely a data entry error.

N3 — Flag biologically implausible counts

`IF count > [species_max_group_size from registry] → flag COUNT_OUTLIER`

Action: Flag. Do not delete. Log: count value, species, expected maximum.

N4 — Distinguish zero-count from missing count

`IF count field is blank → enter MISSING, not 0. IF survey was conducted and species was not detected → enter 0.`

These are analytically different. A zero means surveyed and absent. MISSING means data not recorded. Never substitute one for the other.

OBSERVER AND EFFORT FIELDS

O1 — Map raw observer to Observer Registry

`IF observer_raw not in Observer Registry → flag OBSERVER_UNKNOWN`

Action: Populate observer_code from registry where possible. Preserve observer_raw. Flag unknowns for manual resolution.

O2 — Standardise effort units within dataset

`IF effort recorded in inconsistent units within one dataset → standardise to primary unit`

Action: Convert. Log conversion factor. Do not mix effort units in the same effort_unit column.

Part B — Cross-Dataset Rules (apply after merge)

X1 — Detect exact duplicates

`IF two rows have identical values in: species_code, detection_date, detection_time, site_code, count → flag EXACT_DUPLICATE`

Action: Flag both rows. Do not delete. Requires human decision to retain one or both.

X2 — Detect near-duplicates

`IF two rows match on: species_code, detection_date, site_code AND differ by <30 min in time AND <50m in GPS → flag NEAR_DUPLICATE`

Action: Flag both. Log: the two record IDs and the fields that differ. Do not auto-resolve.

X3 — Detect cross-dataset species name conflicts

`IF same species appears under different codes in different source datasets → verify against Species Registry. Both codes must map to same canonical species.`

Action: Standardise to registry code. Log: all variant codes found, source files, resolution.

X4 — Detect site name conflicts across seasons

`IF same geographic area is referred to by different names in different source files → resolve to canonical site code from Site Registry`

Action: Standardise to registry code. Log: all variant names, source files, resolution.

Worked Cleaning Examples — Before and After

RAW — SPECIES COLUMN

```
Lc
L. catta
Ring-tailed lemur
Lemur catta (juv?)
LEMURCATTA
lm ct
```

HARMONIZED — SPECIES_CODE / SPECIES_LATIN

```
LEMURCATTA / Lemur catta
LEMURCATTA / Lemur catta
LEMURCATTA / Lemur catta
LEMURCATTA / cf. Lemur catta
LEMURCATTA / Lemur catta
LEMURCATTA / Lemur catta
```

RAW — DATE COLUMN (MIXED FORMATS)

```
06/03/24
2024-03-07
March 8, 2024
3/9/24
10.03.2024
11-Mar-24
```

HARMONIZED — DETECTION_DATE (ISO 8601)

```
2024-03-06
2024-03-07
2024-03-08
2024-03-09
2024-03-10
2024-03-11
```

RAW — GPS (MIXED FORMATS)

```
S18°27.456 E47°11.234
-18.457, 47.187
18 27 27 S / 47 11 14 E
18.45, 47.12
UTM 38S 519234 7960123
```

HARMONIZED — LATITUDE / LONGITUDE / FLAG

```
-18.45760 / 47.18723 / VERIFIED
-18.45700 / 47.18700 / VERIFIED
-18.45750 / 47.18722 / VERIFIED
-18.45000 / 47.12000 / LOW_PRECISION
-18.45821 / 47.18734 / VERIFIED
```

SECTION 8

AI Prompt Pack — 12 Prompts

Use with Claude, ChatGPT, or any capable AI assistant. These prompts assist at each stage of the harmonization pipeline. Always verify AI output against your data — AI can produce plausible but wrong suggestions, especially for taxonomic name resolution and coordinate conversion.

Prompt 1 — Master Schema Designer

```
I am harmonizing biodiversity datasets from multiple sources. From my dataset inventory below, design a master schema for the unified dataset. Inventory summary: [Paste your dataset inventory — list of source files with column names, data types, survey methods] For each column in the master schema: - Column name (snake_case, no spaces) - Data type (TEXT / CODE / DATE / TIME / LAT / LON / INT / FLOAT / BOOL / FLAG) - Format specification - Valid values if categorical (controlled vocabulary) - NA handling rule (how to represent missing, illegible, and not-applicable separately) - Whether this column is required or optional Flag any column that will need a controlled vocabulary registry (species, sites, observers).
```

Prompt 2 — Species Name Resolver

```
I have the following species name variants from multiple datasets. For each variant, identify the current accepted Latin binomial, the likely intended species, and any taxonomic notes. Context: - Study region: [country and habitat type] - Target taxa: [mammals / amphibians / reptiles / birds / plants] - Survey period: [date range] For each variant: 1. Most likely intended species (current accepted Latin binomial) 2. Confidence: HIGH / MEDIUM / LOW 3. Alternative interpretation if ambiguous 4. Taxonomic note if name has changed since survey period 5. Recommended species_code (ALLCAPS, underscores, genus+species, max 15 chars) Species variants to resolve: [LIST ONE PER LINE — paste exactly as they appear in your raw data]
```

Prompt 3 — Observer Registry Builder

```
I need to build an Observer Registry for my harmonized dataset. I have the following observer name variants from multiple source files. Help me consolidate them. For each unique observer: 1. List all name variants that appear to refer to the same person 2. Suggest a canonical observer code (initials format — 2-3 capital letters) 3. Flag any ambiguous cases where two variants might be different people 4. Note where the same code might refer to different people in different datasets Observer variants (one per line, with source file): [LIST VARIANTS — e.g., "HFR (file CT_2024)", "Haja (file TRX_2023)", "H.F.Razafindrabe (file VEG_2022)"]
```

Prompt 4 — Coordinate Batch Converter

```
Convert the following GPS coordinates to decimal degrees format (DD.DDDDD), WGS84. Study area bounding box: Latitude: [min_lat] to [max_lat] (Southern Hemisphere: values should be negative) Longitude: [min_lon] to [max_lon] For each coordinate: 1. Convert to decimal degrees 2. Flag any coordinate outside the study area bounding box as SPATIAL_OUTLIER 3. Flag any coordinate with fewer than 4 decimal places as LOW_PRECISION 4. Flag any coordinate where latitude and longitude appear transposed as TRANSPOSED 5. Return: original_value, latitude_dd, longitude_dd, spatial_flag, conversion_notes Coordinates to convert (one per line): [LIST COORDINATES — paste exactly as they appear in your raw data]
```

Prompt 5 — Cleaning Log Generator

```
I have applied the following cleaning operations to my dataset. Generate a structured cleaning log entry for each operation. Dataset ID: [source file ID] Cleaning date: [date] Cleaned by: [name] Operations performed: [Describe each operation, e.g., "Converted 47 date values from DD/MM/YY to YYYY-MM-DD", "Resolved 12 species name variants to 4 canonical species", "Flagged 3 records as SPATIAL_OUTLIER"] For each operation, generate a log entry with: - Operation type - Field(s) affected - Number of records affected - Rule applied (from the cleaning rule library) - Before value example - After value example - Reversibility (can this be reversed from the original data?) - Any records requiring expert review
```

Prompt 6 — Near-Duplicate Detector

```
I have merged biodiversity datasets from multiple sources. Help me identify near-duplicate records. Near-duplicate definition for this project: Two records are near-duplicates if they share: - Same species_code - Same detection_date (or within [X] days if date uncertainty exists) - Same site_code - Detection time within [30] minutes of each other - GPS coordinates within [50] metres of each other From the merged dataset below, identify all candidate near-duplicate pairs. For each candidate pair: 1. Record IDs of both records 2. Fields that match exactly 3. Fields that differ and by how much 4. Recommended action: RETAIN_BOTH / RETAIN_FIRST / RETAIN_SECOND / EXPERT_REVIEW 5. Reasoning Dataset to check (paste as CSV): [PASTE RELEVANT COLUMNS — species_code, detection_date, detection_time, site_code, latitude, longitude, source_file_id, record_id]
```

Prompt 7 — Sensitive Data Flagging

```
I need to apply sensitivity flags to records in my harmonized dataset before sharing externally. Sensitivity rules: 1. Species flagged as IUCN CR or EN in the Species Registry → flag RESTRICTED, offset coordinates by ≥1km in public version 2. Any record with habitat type = [specify sensitive habitat types] → flag RESTRICTED 3. Species in the following list (known poaching targets in this region): [list species] → flag RESTRICTED regardless of IUCN status 4. Records within [distance] of any community settlement → flag COMMUNITY_SENSITIVE, review before sharing For each record in the dataset below: - Apply the appropriate sensitivity flag (PUBLIC / RESTRICTED / COMMUNITY_SENSITIVE / OFFSET_APPLIED) - For RESTRICTED records: calculate an offset coordinate that moves the point ≥1km in a random direction within the same habitat type - Generate a separate "offset log" that maps original to offset coordinates (for internal use only) Dataset to flag (paste as CSV): [PASTE DATASET]
```

Prompt 8 — Darwin Core Mapping

```
Map my harmonized dataset schema to Darwin Core terms for GBIF submission or MRV reporting. My master schema columns: [LIST YOUR COLUMN NAMES AND TYPES] For each column: 1. The equivalent Darwin Core term (or "no direct equivalent") 2. Any format conversion needed for Darwin Core compliance 3. Whether this field is required, recommended, or optional in Darwin Core 4. Any Darwin Core terms I should add to my dataset that are missing from my current schema Also generate: - A meta.xml file template for GBIF submission - A list of the 10 most important Darwin Core terms I should ensure are populated before submission
```

Prompt 9 — Validation Report Generator

```
Generate a data quality validation report for my harmonized biodiversity dataset. Dataset summary: - Total records: [number] - Source datasets: [number and types] - Date range: [earliest] to [latest] - Sites: [number] - Species recorded: [number] Validation results: - Records failing date validation: [number] - Records flagged SPATIAL_OUTLIER: [number] - Records flagged SPECIES_UNKNOWN: [number] - Records flagged NEAR_DUPLICATE: [number] - Records with ILLEGIBLE fields: [number and which fields] - Records flagged RESTRICTED: [number] - Overall data completeness: [% of required fields populated] Generate: 1. A plain-language data quality summary (150 words) 2. A table of issues sorted by severity (High / Medium / Low) 3. Recommendations for which analyses are appropriate at current data quality vs. which require resolving specific issues first 4. A list of issues that can be resolved computationally vs. those requiring expert field knowledge
```

Prompt 10 — R Cleaning Script Generator

```
Generate an R script using tidyverse (dplyr, tidyr, lubridate, sf) that applies the following cleaning operations to my biodiversity dataset. Operations needed: 1. Standardise all dates in column [col_name] from [format] to YYYY-MM-DD 2. Convert GPS coordinates in columns [lat_col, lon_col] from [DMS/DDM] to decimal degrees 3. Map species names in [col_name] to canonical names using a lookup table [describe lookup table structure] 4. Flag records where [col_name] is outside [min, max] as [flag_value] 5. Detect and flag near-duplicates based on [specify criteria] Dataset structure: [Describe column names, types, and a few example rows] The script must: - Include comments explaining each operation - Preserve the original values in separate columns (do not overwrite) - Generate a cleaning log as a dataframe that can be exported as CSV - Be reproducible — the same script on the same input must always produce the same output
```

Prompt 11 — Python Cleaning Script Generator

```
Generate a Python script using pandas, numpy, and pyproj that applies the following cleaning operations to my biodiversity dataset. [Same parameters as Prompt 10 — adapted for Python/pandas syntax] Additional requirements: - Use type annotations throughout - Include a function for each cleaning operation (modular design) - Generate a validation report as a pandas DataFrame at the end - Export the cleaned dataset, the cleaning log, and the validation report as separate CSV files - Include a requirements.txt listing all dependencies
```

Prompt 12 — Harmonization Log Writer

```
Write the harmonization log for a biodiversity data harmonization project based on the following information. Project details: - Project name: [name] - Data manager: [name] - Harmonization date: [date] - Source datasets: [list with file IDs, record counts, date ranges] - Master schema version: [version] - Total records in harmonized dataset: [number] - Records excluded and why: [describe] Cleaning operations summary: [Describe each category of cleaning applied — species names, dates, coordinates, observer codes, etc., with record counts] Outstanding issues (not resolved in this harmonization cycle): [Describe any issues flagged for expert review or requiring additional data] Format: a formal data harmonization log suitable for institutional filing, publication as supplementary material, or MRV audit. Include a version history table at the start.
```

SECTION 9

Six-Point Validation Framework

Run all six checks before the dataset is used in any analysis or shared with any external party. A dataset that fails any check is not ready. Document the results of every check in the harmonization log.

Check 1 — Structural Completeness

WHAT IT TESTS

Are all required columns present? Are all required fields populated? Are all column types correct?

`Pass criteria: 0 blank values in required columns. 0 type violations. 0 missing required columns.`

- → List every required column from the master schema.
- → Count blank values in each required column. Blank is a fail — ILLEGIBLE and MISSING are acceptable.
- → Verify that each column contains the correct data type (no text in numeric columns, no numeric values in FLAG columns).

Check 2 — Controlled Vocabulary Compliance

WHAT IT TESTS

Do all CODE and FLAG fields contain only values from the defined controlled vocabularies?

`Pass criteria: 0 values in any CODE or FLAG column that are not in the defined vocabulary.`

- → For each CODE column: check that every value appears in its registry (species, site, observer).
- → For each FLAG column: check that every value is from the defined flag vocabulary.
- → A value of SPECIES_UNKNOWN is compliant — an undefined species abbreviation is not.

Check 3 — Spatial Validity

WHAT IT TESTS

Are all GPS coordinates within the study area bounding box? Are they in the correct format and range?

`Pass criteria: 0 unresolved SPATIAL_OUTLIER flags. 0 coordinates outside valid range (lat -90 to 90, lon -180 to 180).`

- → Verify every coordinate is within the study area bounding box or is explicitly flagged as SPATIAL_OUTLIER.
- → Check for transposed coordinates (latitude in longitude range).
- → Verify all coordinates are decimal degrees — no DMS, no UTM in the final dataset.

Check 4 — Temporal Validity

WHAT IT TESTS

Are all dates within the known survey period? Are date formats consistent?

`Pass criteria: 0 dates outside the overall survey period (or all outliers explicitly flagged). 100% of dates in YYYY-MM-DD format.`

- → Verify all dates fall within [earliest survey start] and [latest survey end], or are flagged TEMPORAL_OUTLIER.
- → Check for common conversion errors: dates that read correctly in one format but mean a different day in another (e.g., 03/04 = April 3 or March 4).

Check 5 — Effort Coverage

WHAT IT TESTS

Does every detection record have a corresponding effort record? Is the effort table complete?

`Pass criteria: 0 detection records with a NULL effort_link_id. Effort coverage ≥95% of expected survey days/stations.`

- → Join the detection table to the effort table on effort_link_id. Flag any detections with no matching effort record.

- → Calculate effort coverage: (actual survey days or stations) ÷ (planned survey days or stations). Flag if below 95%.
- → Verify that effort units are consistent within each survey method.

Check 6 — Sensitivity Compliance

> WHAT IT TESTS
>
> Have all sensitive records been correctly identified and handled before any external sharing?
>
> `Pass criteria: 0 IUCN CR or EN species records with PUBLIC sensitivity flag in the shareable version. 0 sensitive coordinates in the shareable version without OFFSET_APPLIED flag.`

- → Join the Species Registry to the dataset. Verify that every CR and EN species has sensitivity_flag = RESTRICTED or OFFSET_APPLIED.
- → Verify that the shareable (public) version of the dataset does not contain exact coordinates for any RESTRICTED record.
- → Verify that the offset log is maintained separately and is not included in the shareable dataset.

SECTION 10

Harmonization Log Template

BIODIVERSITY DATA HARMONIZATION LOG Version: 1.0 | Date: [YYYY-MM-DD] | Data Manager: [name] Project: [project name] | Institution: [institution] ── SOURCE DATASETS ── Dataset ID | Method | Records | Date Range | Known Issues [CT_IHOFA_2024] | Camera trap | [1,247] | 2024-03-01–05-31 | [GPS format mixed] [TRX_IHOFA_2024]| Transect | [487] | 2024-03-05–04-20 | [Observer codes unclear] [ACU_IHOFA_2024]| Acoustic | [892] | 2024-03-01–05-15 | [Date format DD/MM/YY] ── MASTER SCHEMA ── Schema version: [1.0] | Schema file: [filename] Total columns in master schema: [N] | Required columns: [N] ── REGISTRIES USED ── Species Registry: [filename] | Species count: [N] | Unresolved species: [N] Site Registry: [filename] | Sites: [N] Observer Registry: [filename] | Observers: [N] | Unresolved observers: [N] ── CLEANING OPERATIONS APPLIED ── Rule | Records affected | Action taken | Log file D1 | [47 records] | Date format standardised from DD/MM/YY to YYYY-MM-DD | [file] G1 | [23 records] | DMS coordinates converted to decimal degrees | [file] S1 | [156 records] | Species names resolved via Species Registry | [file] S4 | [3 records] | Taxonomic name updated to current accepted name | [file] N1 | [8 records] | Count ranges split to count_min / count_max | [file] X1 | [4 records] | Exact duplicates flagged — expert review pending | [file] X2 | [11 records] | Near-duplicates flagged — expert review pending | [file] ── RECORDS EXCLUDED ── Reason | Records excluded | Source file Survey outside project area | [5] | [CT_IHOFA_2024] Date pre-dates survey start | [2] | [TRX_IHOFA_2024] ── OUTSTANDING ISSUES (EXPERT REVIEW REQUIRED) ── Issue | Records | Priority | Assigned to Exact duplicate resolution | 4 | HIGH | [name] SPECIES_UNKNOWN resolution | 7 | MEDIUM | [taxon expert name] Near-duplicate resolution | 11 | MEDIUM | [name] ── VALIDATION RESULTS ── Check 1 (Structural completeness): PASS — 0 blank required fields Check 2 (Vocabulary compliance): PASS — 0 invalid codes Check 3 (Spatial validity): PASS — 3 SPATIAL_OUTLIER records retained+flagged Check 4 (Temporal validity): PASS Check 5 (Effort coverage): PASS — 97.3% effort coverage Check 6 (Sensitivity compliance): PASS — 47 records flagged RESTRICTED in public version ── FINAL DATASET ── Harmonized records: [2,619] | Excluded: [7] | Flagged for review: [22] Public (shareable) version: [filename] | Internal (full) version: [filename] Sensitivity offset log: [filename — INTERNAL ONLY — never share]

SECTION 11

R and Python Code Templates

Ready-to-adapt code for the most common harmonization operations. Replace the placeholder column names and values with your actual schema.

R Template — Date Standardisation

R · TIDYVERSE · DATE STANDARDISATION

```
# Date standardisation — multiple input formats to YYYY-MM-DD library(tidyverse)
library(lubridate) standardise_date <- function(date_col) { # Try multiple formats in order
of likelihood formats <- c("%Y-%m-%d", "%d/%m/%Y", "%m/%d/%Y", "%d/%m/%y", "%d.%m.%Y", "%B
%d, %Y") parsed <- parse_date_time(date_col, orders = formats, quiet = TRUE) format(parsed,
"%Y-%m-%d") # Return as character ISO 8601 } df <- df |> mutate( detection_date_raw =
detection_date, # Preserve original detection_date = standardise_date(detection_date),
date_flag = if_else(is.na(detection_date), "DATE_PARSE_FAIL", "OK") )
```

R Template — Coordinate Validation and Flagging

R · SPATIAL VALIDATION · STUDY AREA BOUNDING BOX

```
# Define study area bounding box lat_min <- -20.0 ; lat_max <- -14.0 lon_min <- 44.0 ;
lon_max <- 50.5 df <- df |> mutate( lat_raw = latitude, lon_raw = longitude, # Preserve
originals latitude = as.numeric(latitude), longitude = as.numeric(longitude), spatial_flag
= case_when( is.na(latitude) | is.na(longitude) ~ "MISSING", latitude < lat_min | latitude
> lat_max ~ "SPATIAL_OUTLIER", longitude < lon_min | longitude > lon_max ~
"SPATIAL_OUTLIER", nchar(sub(".*\\.", "", as.character(latitude))) < 4 ~ "LOW_PRECISION",
TRUE ~ "VERIFIED" ) )
```

Python Template — Species Name Resolution

PYTHON · PANDAS · SPECIES NAME LOOKUP AGAINST REGISTRY

```
import pandas as pd # Load species registry registry = pd.read_csv('species_registry.csv') lookup = dict(zip(registry['species_raw_variant'], registry['species_code'])) latin_lookup = dict(zip(registry['species_raw_variant'], registry['species_latin'])) # Apply lookup to dataset df['species_raw'] = df['species'].copy() # Preserve original df['species_code'] = df['species'].map(lookup) df['species_latin'] = df['species'].map(latin_lookup) # Flag unresolved names df['species_flag'] = df['species_code'].apply( lambda x: 'SPECIES_UNKNOWN' if pd.isna(x) else 'RESOLVED' ) # Log unresolved for manual review unresolved = df[df['species_flag'] == 'SPECIES_UNKNOWN']['species_raw'].unique() pd.DataFrame({'unresolved_species': unresolved}).to_csv('unresolved_species_log.csv', index=False)
```

Python Template — Duplicate Detection

PYTHON · PANDAS · NEAR-DUPLICATE DETECTION

```
from itertools import combinations import numpy as np def haversine_m(lat1, lon1, lat2, lon2): """Distance in metres between two GPS points.""" R = 6371000 phi1, phi2 = np.radians(lat1), np.radians(lat2) dphi = np.radians(lat2 - lat1) dlam = np.radians(lon2 - lon1) a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2)*np.sin(dlam/2)**2 return R * 2 * np.arctan2(np.sqrt(a), np.sqrt(1-a)) near_dupes = [] for i, j in combinations(df.index, 2): r1, r2 = df.loc[i], df.loc[j] if (r1['species_code'] == r2['species_code'] and r1['detection_date'] == r2['detection_date'] and abs((r1['detection_time'] - r2['detection_time']).total_seconds()) < 1800): dist = haversine_m(r1.latitude, r1.longitude, r2.latitude, r2.longitude) if dist < 50: near_dupes.append({'record_1': r1.record_id, 'record_2': r2.record_id, 'dist_m': round(dist, 1)})
```

SECTION 12

Quick Start — Harmonize Your First Dataset Pair

Before You Begin

- ☐ Identify your two or more source datasets. Start with the simplest pair — same method, same site, two different seasons.
- ☐ Complete the Dataset Inventory template for each source file (Step 1).
- ☐ Open a new spreadsheet. Create three tabs: Species Registry, Site Registry, Observer Registry.

Hours 1–2 — Schema and Registries

- ☐ Use Prompt 1 to draft the master schema from your inventory. Review and adjust.
- ☐ Use Prompt 2 to help resolve species name variants. Verify every suggestion against a regional taxonomy reference.
- ☐ Use Prompt 3 to build the Observer Registry.
- ☐ Save all three registries as CSV files.

Hours 3–5 — Individual Dataset Cleaning

- ☐ Open the first source dataset. Create a copy — never clean the original.
- ☐ Apply the date cleaning rules (D1–D4) using the R or Python templates from Section 11.
- ☐ Apply the coordinate cleaning rules (G1–G6). Use Prompt 4 for batch conversion.
- ☐ Apply the species name rules (S1–S4) using the Python template.
- ☐ Rename all columns to match the master schema. Log every rename in the column mapping table.
- ☐ Repeat for the second source dataset.

Hours 6–7 — Merge and Cross-Dataset Cleaning

- ☐ Merge the two cleaned datasets (append rows). Add source_file_id, source_row, and merge_date columns.

- ☐ Use Prompt 6 to detect near-duplicates. Flag them — do not delete.
- ☐ Run the cross-dataset species and site conflict checks (Rules X3, X4).
- ☐ Apply sensitivity flags using the Species Registry (Step 7 / Prompt 7).

Hour 8 — Validation and Documentation

- ☐ Run all six validation checks from Section 9. Document results for each check.
- ☐ Use Prompt 9 to generate the validation report.
- ☐ Use Prompt 12 to write the harmonization log.
- ☐ Export: harmonized dataset (internal version), public (sensitivity-offset) version, and the harmonization log.

Minimum Viable vs. Premium Version

Feature	This Version (MVP)	Premium (Future)
8-step harmonization pipeline	✓	✓
Master schema design guide	✓	✓
Schema library (6 data types)	✓	✓ Expanded to 10 types
40 cleaning rules	✓	✓ 60 rules
12 AI prompts	✓	✓ 20 prompts
Six-point validation framework	✓	✓
R and Python code templates (5)	✓	✓ 15 templates
Harmonization log template	✓	✓
Darwin Core mapping guide	Prompt only	Full mapping table + meta.xml template
Pre-built Google Sheets template with validation rules	—	Planned
GBIF submission walkthrough	—	Planned
MRV-grade data package builder	—	Planned (bridges to ERIS)
French version (full translation)	—	Planned

OPLURIX

Unharmonized data is not data. *It is potential — waiting for a decision to be made.*

Every dataset integration decision is a scientific decision. When you document those decisions — what was merged, what was cleaned, what was excluded and why — you transform a dataset into evidence. Evidence that can be verified, audited, shared, and built upon. That is what conservation finance infrastructure requires. That is what ERIS is built on.

Biodiversity Data Harmonizer v1.0

An OPLURIX Product · 2026

marvelous-valkyrie-f94b2f.netlify.app